



UNIVERSITÄT
LEIPZIG

8. Vorlesung Evaluierung

INTERACTIVE VISUAL DATA MINING

Übersicht

8. Evaluierung

- 8.1 Einführung in die Evaluierung
- 8.2 Datenaufteilung
- 8.3 Evaluierung von Klassifikatoren
- 8.4 Evaluierung numerischer Vorhersagen

Lernziele:

- Wissen, warum Evaluierung nötig ist, welche Probleme auftreten und wie Experimente aufgebaut werden.
- Verstehen warum Datenaufteilung notwendig ist, sowie Validierungsverfahren und deren Vor- und Nachteile kennen
- Hyperparameter von Modellparametern unterscheiden; HPO-Verfahren kennen; typische Loss-Kurven diagnostizieren.
- Klassifikatoren und Regressionsmodelle evaluieren und passende Metriken interpretieren können.

EVALUIERUNG

– Warum?

- Unterschiedliche Verfahren können auf denselben Daten sehr unterschiedliche Ergebnisse erzeugen
- Systematischer Ansatz zur Bewertung von Methoden/Modellen
- Ziel: Leistungsfähigkeit von Modellen fair, reproduzierbar und vergleichbar messen

– Wie?

- Führe Experimente durch
- Vergleich von Modellen anhand definierter Metriken

– Probleme:

- Begrenzte oder verzerrte Datenqualität
- Overfitting auf Trainingsdaten
- Unterschiedliche Modelle optimieren unterschiedliche Zielgrößen
- Ergebnisse sind abhängig von:
 - Datenaufteilung (Split)
 - Metrikwahl
 - Zufälligkeit (Initialisierung, Sampling)

EVALUIERUNG

- Training und Tests
 - Die Performanz für Klassifizierungsprobleme wird mit der Fehlerrate gemessen
 - Fehlerrate: Anzahl der falschen Vorhersagen über alle Vorhersagen
 - Ziel: Erreiche gute Vorhersagen auf **neuen** Daten
- Die Fehlerrate auf dem Trainingsdatensatz eignet sich nicht als guter Indikator
 - Wird auch *Resubstitution Fehler* genannt
 - Ist nur ein erster Indikator für die Qualität des Modells
- Teile den Datensatz in
 - Trainingsdatensatz
 - Validierungsdatensatz
 - Testdatensatz

EVALUIERUNG

- Training und Tests
 - Nutzung von verschiedenen, unabhängigen Datensätzen ist oft sinnvoll:
 - Trainingsdatensatz: Erstelle die Regeln (Parameter)
 - Validierungsdatensatz:
 - Modelloptimierung und Auswahl
 - Hyperparameter-Tuning
 - Testdatensatz: Test der Performanz des finalen und optimierten Models
 - Testdaten müssen strikt vom Trainings- und Validierungsprozess getrennt werden, sonst kommt es zu Data Leakage und einer zu optimistischen Leistungsabschätzung
 - Große Datensätze sind hier hilfreich
 - Bei kleinen Datensätzen fehlen oft genügend bereits gelabelte Beispiele. Deshalb müssen Menschen die Daten häufig selbst manuell annotieren
 - jedes Beispiel zählt stark → schlechte Labels wirken sich stärker aus

EVALUIERUNG

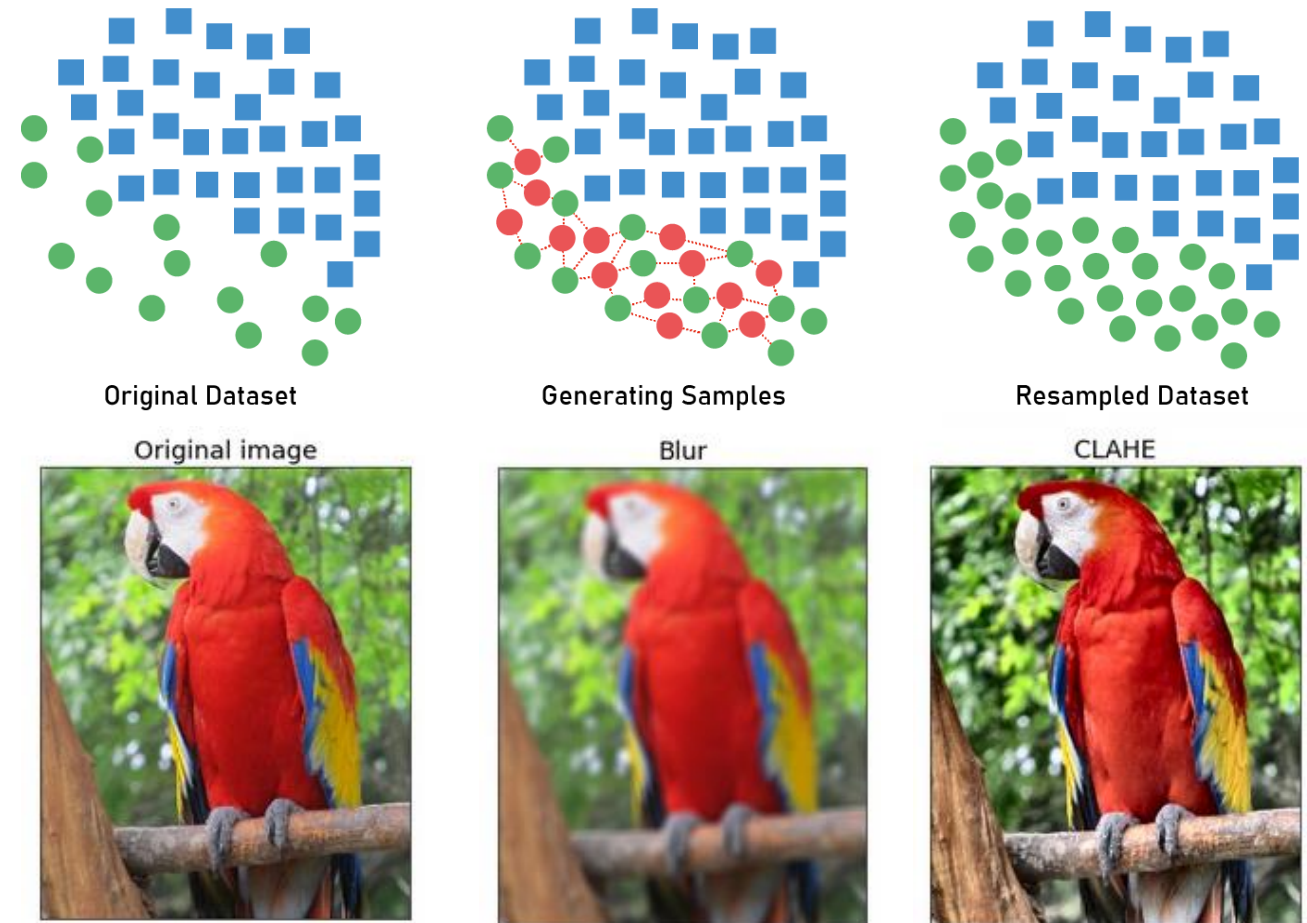
- Data Augmentation
 - Verfahren zur künstlichen Erweiterung eines Datensatzes
 - Neue Trainingsdaten werden aus bestehenden Daten erzeugt
 - Ziel: mehr Vielfalt ohne zusätzliche Datenerhebung
- Reduziert Overfitting und verbessert Modellrobustheit
- Erzeugte Daten sind keine echten neuen Informationen und zu starke Transformationen können Labels verfälschen
- Nicht jede Domäne ist sinnvoll augmentierbar

EVALUIERUNG

– Data Augmentation

– Synthetic Minority Over-sampling Technique (SMOTE)

- Erzeugt synthetische Beispiele für seltene Klassen durch Interpolation zwischen ähnlichen Datenpunkten
- Rauschen hinzufügen
- Bilder
 - Drehen, spiegeln, skalieren, Helligkeit / Kontrast verändern, zuschneiden oder leichtes Rauschen hinzufügen



EVALUIERUNG

– Hyperparameter

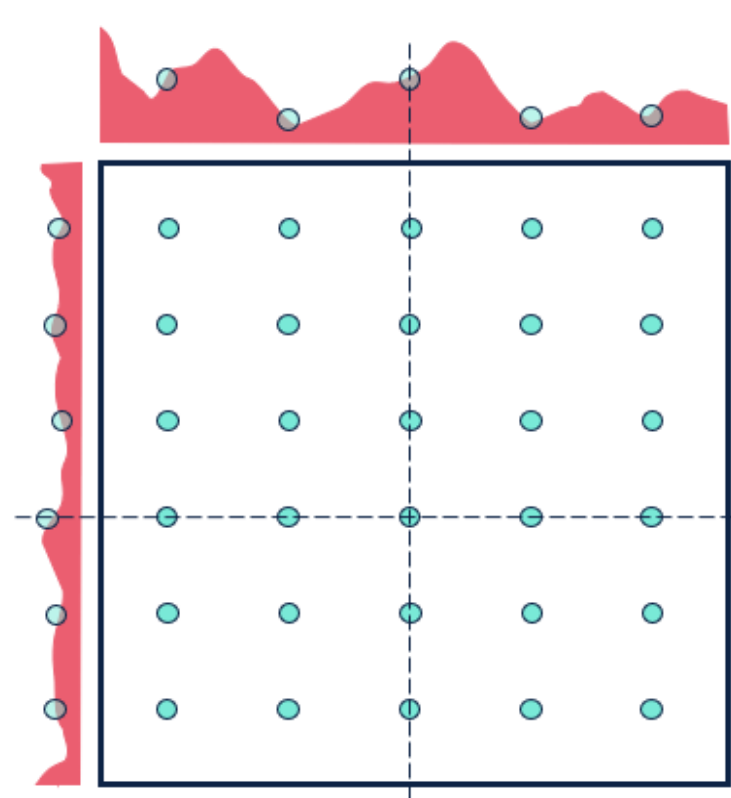
- Parameter, die vor dem Training festgelegt werden und das Lernverhalten eines Modells steuern.
- Unterschied zu Modelparameter (z. B. Gewichte in neuronalen Netzen, Regeln, oder Trennebenen), die während des Trainings gelernt werden.
- Ziel: Optimale Kombination von Hyperparametern finden, um Performance zu maximieren und Overfitting/Underfitting zu vermeiden

– Beispiele

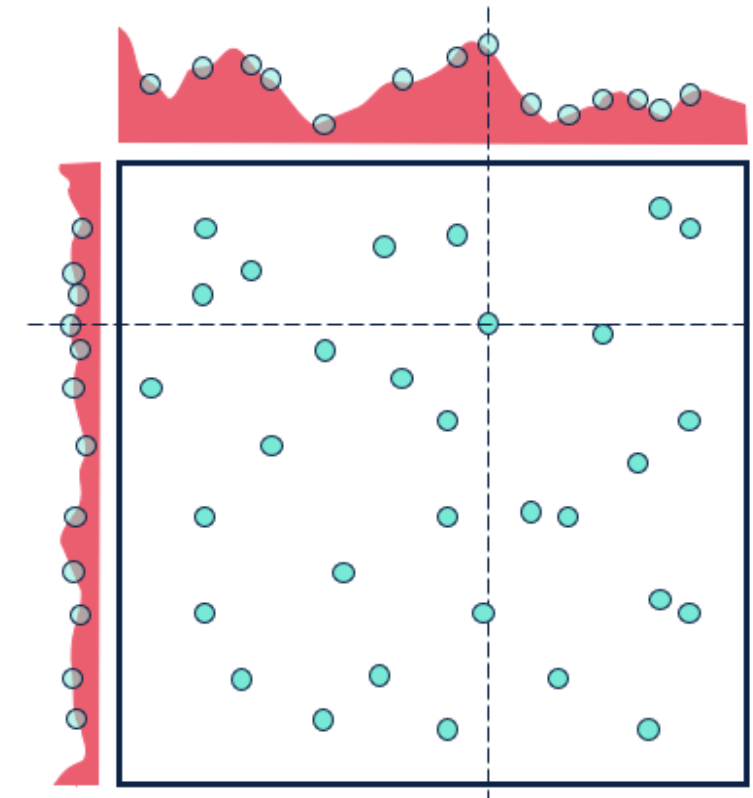
- SVM: Kernel oder Soft-margin C
- Entscheidungsbäume: maximale Tiefe
- Neuronale Netze:
 - Learning Rate: Schrittweite beim Update der Gewichte
 - Batch Size: Anzahl der Samples pro Trainingsschritt
 - Anzahl Hidden Layers und Neuronen
 - Activation Functions: ReLU, Sigmoid, Tanh
 - Und viele mehr

EVALUIERUNG

- Hyperparameter Optimierung (HPO)
 - Finde die optimale Kombination von Hyperparametern aus einem gegebenen Suchraum
 - Balance zwischen Performance und Rechenaufwand
- Grid Search: Kombinationen von Hyperparametern werden systematisch ausprobiert
 - Sehr gründlich, aber extrem teuer bei vielen Parametern
 - Schrittweite bei kontinuierlichen Werten?
- Random Search: Zufällige Auswahl von Kombinationen innerhalb des Suchraums



Grid Search

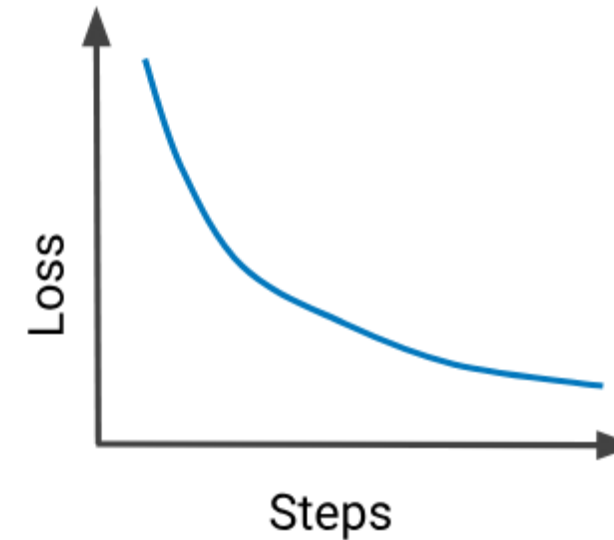


Random Search

Bergstra, J., & Bengio, Y. (2012). *Random search for hyper-parameter optimization*.
Journal of Machine Learning Research

EVALUIERUNG

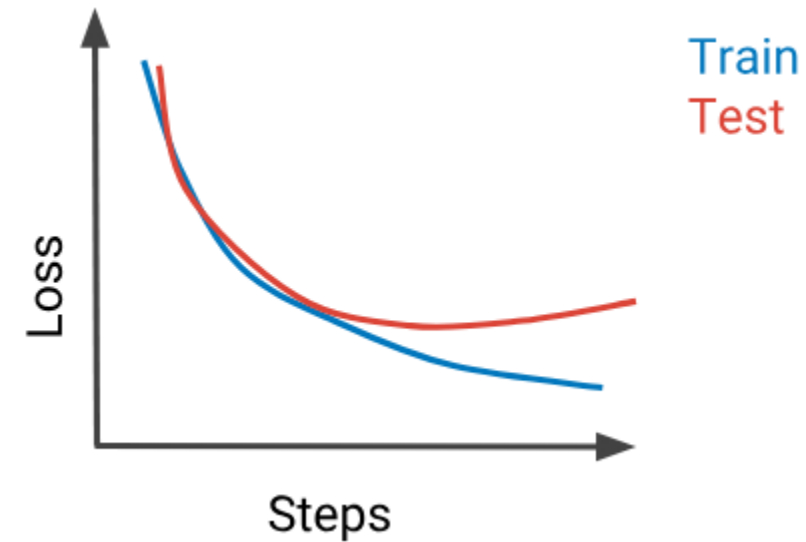
- Loss (Verlustfunktion)
- Der Loss (Verlust) ist eine Funktion, die misst, wie gut oder schlecht ein Modell Vorhersagen trifft
- Vergleich zwischen Vorhersage und tatsächlichem Wert
- Ziel im Machine Learning: Minimierung des Loss → bessere Modellleistung



- Eine ideale Loss-Funktion, der Loss sinkt durchgehend mit mehr Trainingsschritten

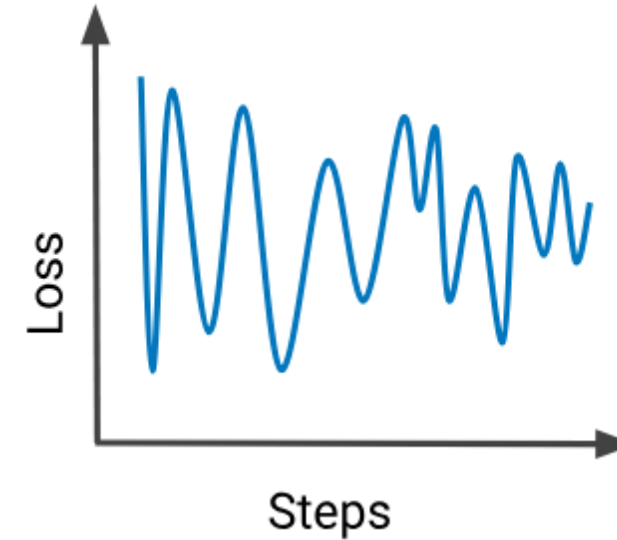
EVALUIERUNG

- Test-Loss weicht vom Trainings-Loss ab
 - Das Modell zeigt Overfitting auf den Trainingsdaten
 - Es generalisiert nicht ausreichend auf unbekannte Daten
- Lösungen
 - Modell vereinfachen
 - Anzahl der Features/Attribute in den Daten reduzieren
 - Sicherstellen, dass Trainings- und Testdaten statistisch vergleichbar sind



EVALUIERUNG

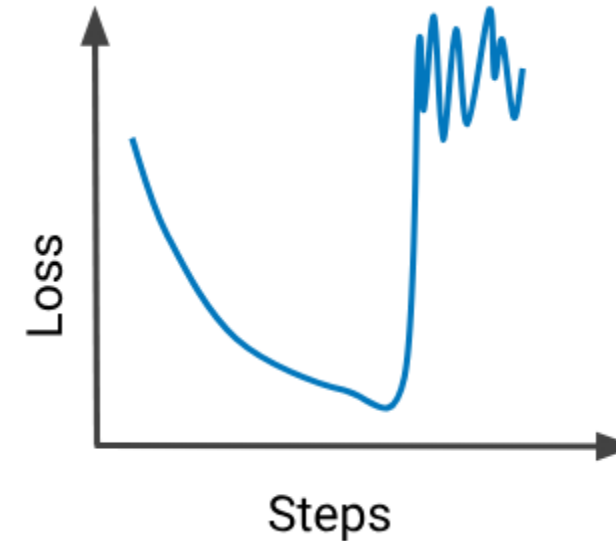
- Oszillierende Losskurve
 - Lernrate ist vermutlich zu hoch
 - Das Training konvergiert nicht stabil
- Lösungen
 - Lernrate reduzieren
 - verbessert Stabilität und Konvergenz
 - Datenqualität prüfen
 - fehlerhafte oder inkonsistente Beispiele entfernen
 - verbessert generell die Trainingsstabilität



- Trainingssatz zunächst stark reduzieren
 - nur wenige, sehr vertrauenswürdige Beispiele verwenden
- Ziel: stabile Konvergenz auf kleinem Datensatz erreichen. Danach schrittweise erweitern
 - problematische Beispiele identifizieren, die Oszillationen verursachen

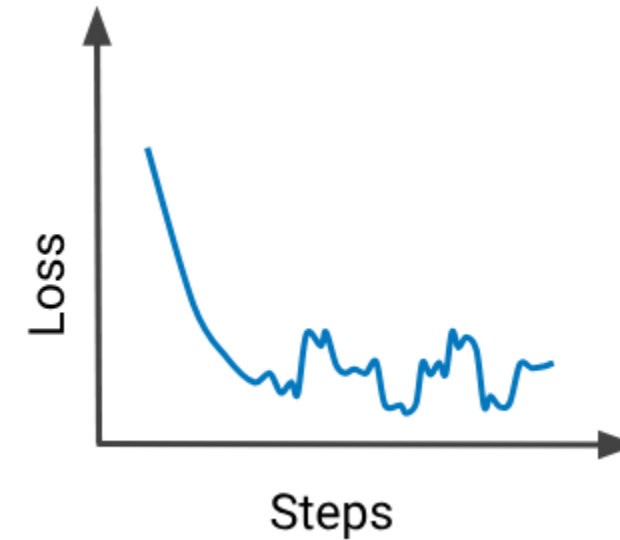
EVALUIERUNG

- Starker Anstieg der Losskurve
 - Mögliche Ursache: Ausreißer in den Eingabedaten
→ einzelne Batches können durch schlechte Mischung stark verzerrt sein
 - Mögliche Ursache: NaN-Werte in den Daten
→ z. B. durch Division durch 0 oder fehlerhafte Vorverarbeitung



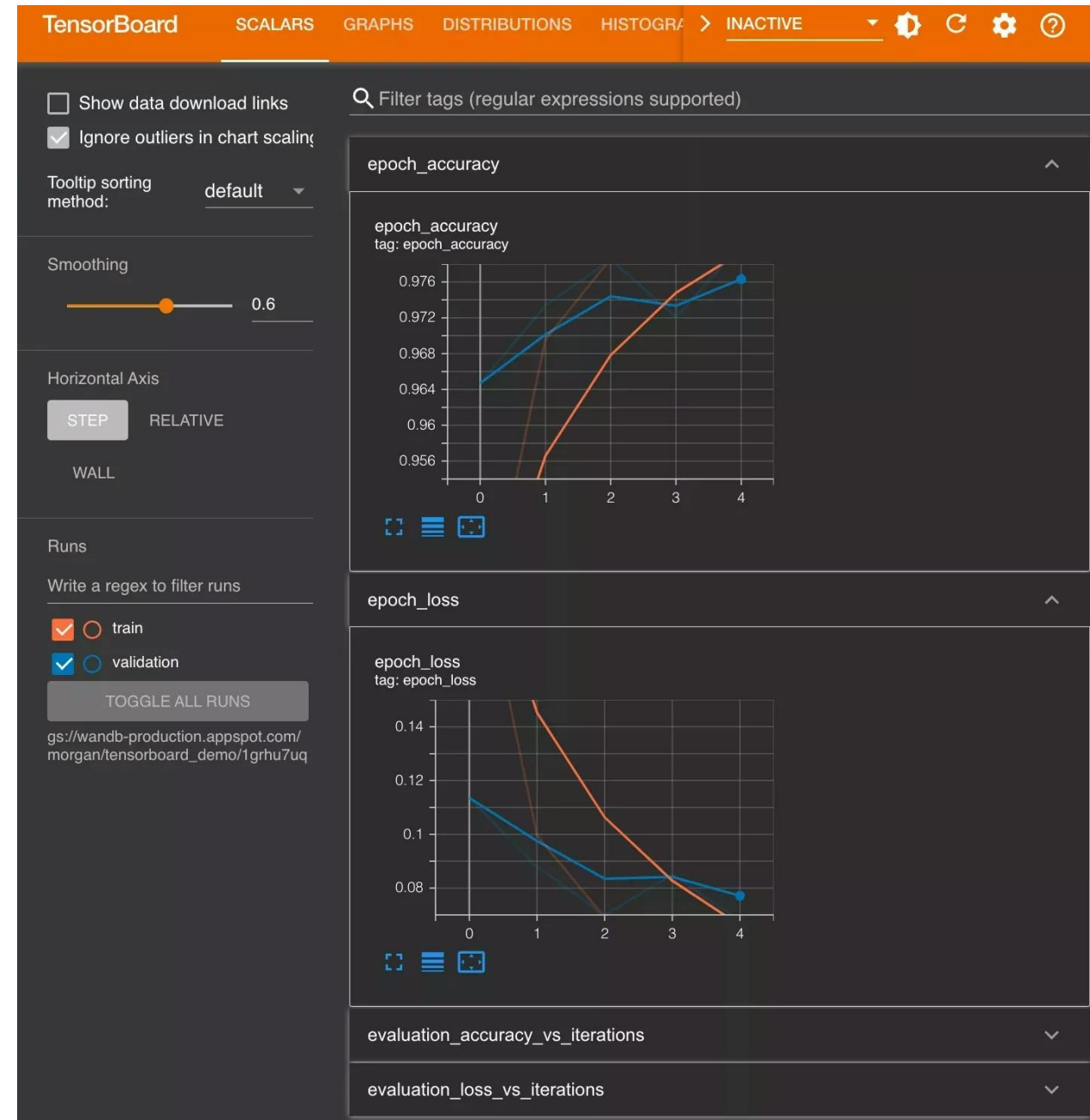
EVALUIERUNG

- Loss-Kurve bleibt hängen
 - Trainingsdaten sind nicht ausreichend gemischt
 - Modell sieht Beispiele in ungünstiger Reihenfolge (erst Klasse A dann B)
 - führt zu instabilem oder stagnierendem Lernverlauf
- Lösung
 - Trainingsdaten besser durchmischen
 - zufällige Reihenfolge sicherstellen (Shuffle)



EVALUIERUNG

- TensorBoard: Tool zur Überwachung von Trainingsprozessen
- Visualisiert z. B.
 - Loss- und Accuracy-Kurven
 - Lernrate-Verlauf
 - Gewichtsverteilungen
 - Gradientenverhalten
- Frühes Erkennen von Problemen (Overfitting, Oszillation)
- Vergleich mehrerer Trainingsläufe



EVALUIERUNG

- Vorhersage Performanz
 - Hängt von der Größe des Testdatensatzes ab
 - Qualität des Klassifizierers hängt vom Trainingsdatensatz ab
 - Nutze Statistik: **Bernoulli-Kette**
 - Abfolge von unabhängigen Versuchen
 - Beispiel: Münzwurf
- Angenommen, die Münze hat einen Bias
 - Führe eine Reihe von Experimenten durch
 - Sage immer Kopf vorher
 - Messen nach N Versuchen eine 75% Erfolgsrate (Vorhersage Kopf, Ergebnis Kopf)
 - Was ist dann die richtige Erfolgsrate p ?

EVALUIERUNG

- Vorhersage Performanz
 - Einzelner Bernoulli Versuch
 - Mittelwert: p
 - Varianz: $p \cdot (1 - p)$
 - N Bernoulli Versuche
 - Mittelwert: p
 - Varianz: $\frac{p \cdot (1 - p)}{N}$
- Wie schaut die Verteilung für große N aus?
 - Es nähert sich der Normalverteilung an

EVALUIERUNG

- Vorhersage Performanz

- Nimm eine zufällige Variable X an

- Mittelwert: 0

- Varianz: 1

- $\Pr[-z \leq X \leq z] = c$

- Es gibt Tabellen für die Werte z und c

- Angenommen, die Daten sind normalverteilt:

- Für einseitige Tests sehen die Tabellen so aus : $\Pr[X \geq z]$

- Für Normalverteilungen gilt: $\Pr[X \geq z] = \Pr[X \leq z]$

- $\Pr[X \geq z] = 5\%$: 5% Chance, dass $X > 1.65 \cdot \sigma + \mu$

- Äquivalent zu: $\Pr[-1.65 \leq X \leq 1.65] = 90\%$

- Finale Gleichung: $\Pr\left[-z < \frac{f-p}{\sqrt{\frac{p \cdot (1-p)}{N}}} < z\right] = c$

EVALUIERUNG

- Vorhersage Performanz

- Konfidenzgrenzen:

- $$p = \frac{f + \frac{z^2}{2N} \pm z \sqrt{\frac{f}{N} - \frac{f^2}{N} + \frac{z^2}{4N^2}}}{1 + \frac{z^2}{N}}$$

- Erhalte von c (Konfidenz) z aus der Tabelle

- Nutze die Formel mit:

- Erfolgsrate $f = \frac{S}{N}$

- S: Anzahl der Erfolge

- N: Anzahl der Versuche

- z: siehe Tabelle

EVALUIERUNG

- Vorhersage Performanz

- $f = 75\%, N = 1000, c = 80\%$ ergibt $[0.732, 0.767]$

- $f = 75\%, N = 100, c = 80\%$ ergibt $[0.691, 0.801]$

- Erinnerung:

- Die Annahme einer Normalverteilung ist nur bei großen N schlüssig!

EVALUIERUNG

- *Holdout* Methode

- Datenmengen für das Lernen und Testen sind oft limitiert verfügbar
- Reserviere vorab einen Teil der Daten für das Testen
- Nutzung der restlichen Daten für das Training

- Manchmal müssen auch Daten für die Validierung reserviert werden
- Oft:
 - 70 % Training
 - 15 % Validierung
 - 15 % Test
- Oder 80/10/10
- Oder 60/20/20
- Stark abhängig von zufälligem Split: Ergebnis kann zufällig gut oder schlecht sein

EVALUIERUNG

– *Holdout* Methode

- Problem: Die Datensätze können nicht repräsentativ sein
- Im Trainingsdatensatz fehlt eine Klasse
- Eine Klasse ist im Trainingsdatensatz unterrepräsentiert

– Lösung: Stratified Holdout

- zufällige Aufteilung der Daten unter Erhalt der Klassenverteilung
- Trainings- und Testdatensatz enthalten proportionale Repräsentation aller Klassen
- verhindert Verzerrungen durch zufälliges Sampling bei unbalancierten Daten
- keine Garantie für perfekte Repräsentation seltener Klassen, nur proportionaler Erhalt
- Immer noch nur ein Split: weiterhin hohe Varianz möglich

EVALUIERUNG

- Repeated Holdout
 - Wiederhole die Analyse (mit Stratifikation) mehrere Male
 - Wähle jedes Mal eine Teilmenge der Daten zufällig für das Testen
 - Nutze die restlichen Daten als Trainingsdatensatz
- Zusammengefasste Fehlerrate:
 - Bilde den Durchschnitt über alle einzelnen Fehlerraten
 - Nicht vollständig kontrolliert: Manche Datenpunkte werden evtl. oft getestet, andere selten

EVALUIERUNG

- k-fold Kreuzvalidierung
 - Erweiterung der Holdout Methode
 - Wähle eine feste Anzahl an *folds* (Partitionen) für die Daten
 - Normalerweise sind alle folds gleich groß
- Jeder fold wird einmal für das Testen genutzt
- Die übrigen werden für das Training genutzt
- Bei drei folds:
 - Dreifache Kreuzvalidierung
 - Stratifizierte dreifache Kreuzvalidierung
- Bilde den Durchschnitt der Fehlerraten

EVALUIERUNG

- k-fold Kreuzvalidierung

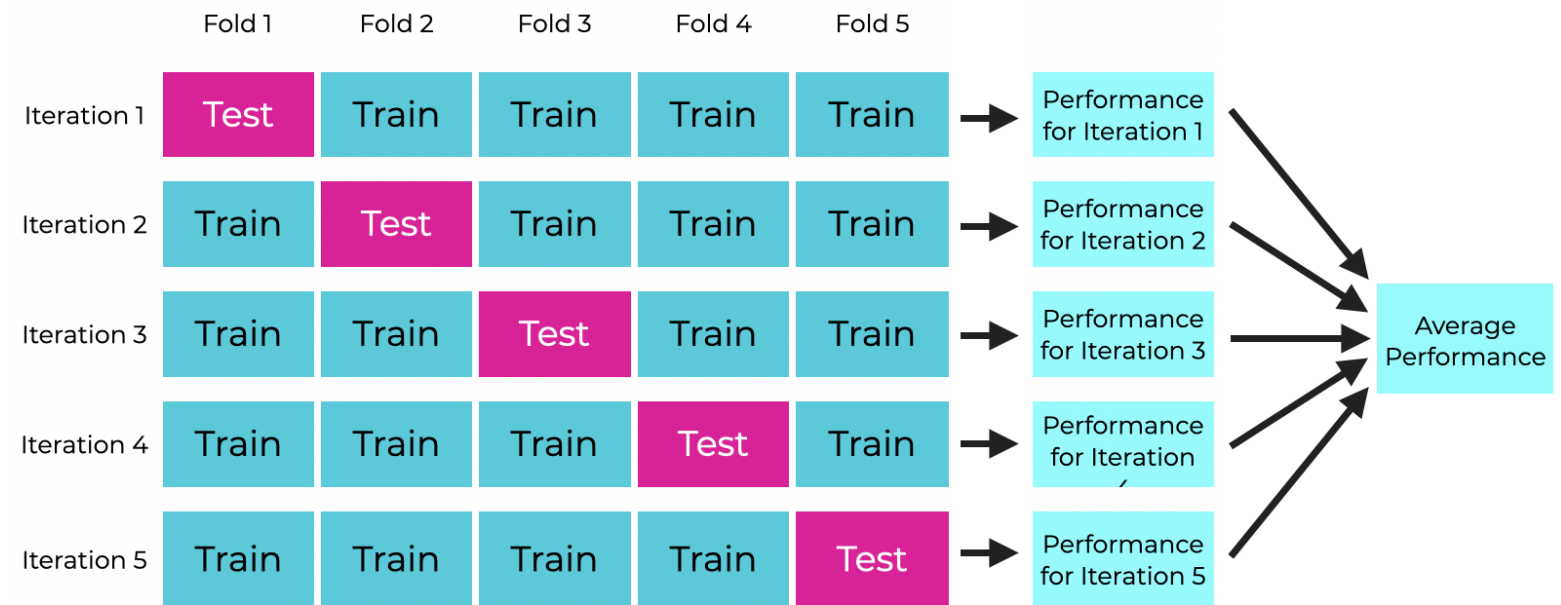
- Standard: 10 folds

- Warum 10?

- Praktische Erfahrungen

- Theoretische Nachweise

- Muss dennoch nicht als
“Standardwert” angenommen werden!



EVALUIERUNG

- Leave-one-out Kreuzvalidierung (LOOCV)

- Datensatz: n Instanzen

- n -fold Kreuzvalidierungen

- Lasse jede Instanz nacheinander aus

- Nutze $n-1$ Instanzen für das Training

- Nutze eine Instanz für das Testen

- Bilde den Durchschnitt über alle n Bewertungen



EVALUIERUNG

– Leave-one-out Kreuzvalidierung (LOOCV)

– Vorteile:

- Nutzung der maximal möglichen Datenmenge für das Training
- Dadurch kann es zur Steigerung der Klassifikatorperformanz kommen
- Deterministisch (kein random sampling notwendig)
- Optimal für kleinere Datensätze

– Nachteile

- Rechenzeit!
- Fast unmöglich für sehr große Datensätze
- Kann nicht stratifiziert werden
 - Jeder Fold hat gröÙe 1
 - Garantiert nicht stratifiziertes Ergebnis
- LOOCV kann bei kleinen, balancierten Datensätzen Fehler stark überschätzen
 - Annahme: Datensatz mit gleicher Anzahl von Instanzen für 2 Klassen
 - Berechne die Mehrheitsklasse:
 - Echte Fehlerrate: 50%
 - Berechnete Fehlerrate: 100%

EVALUIERUNG

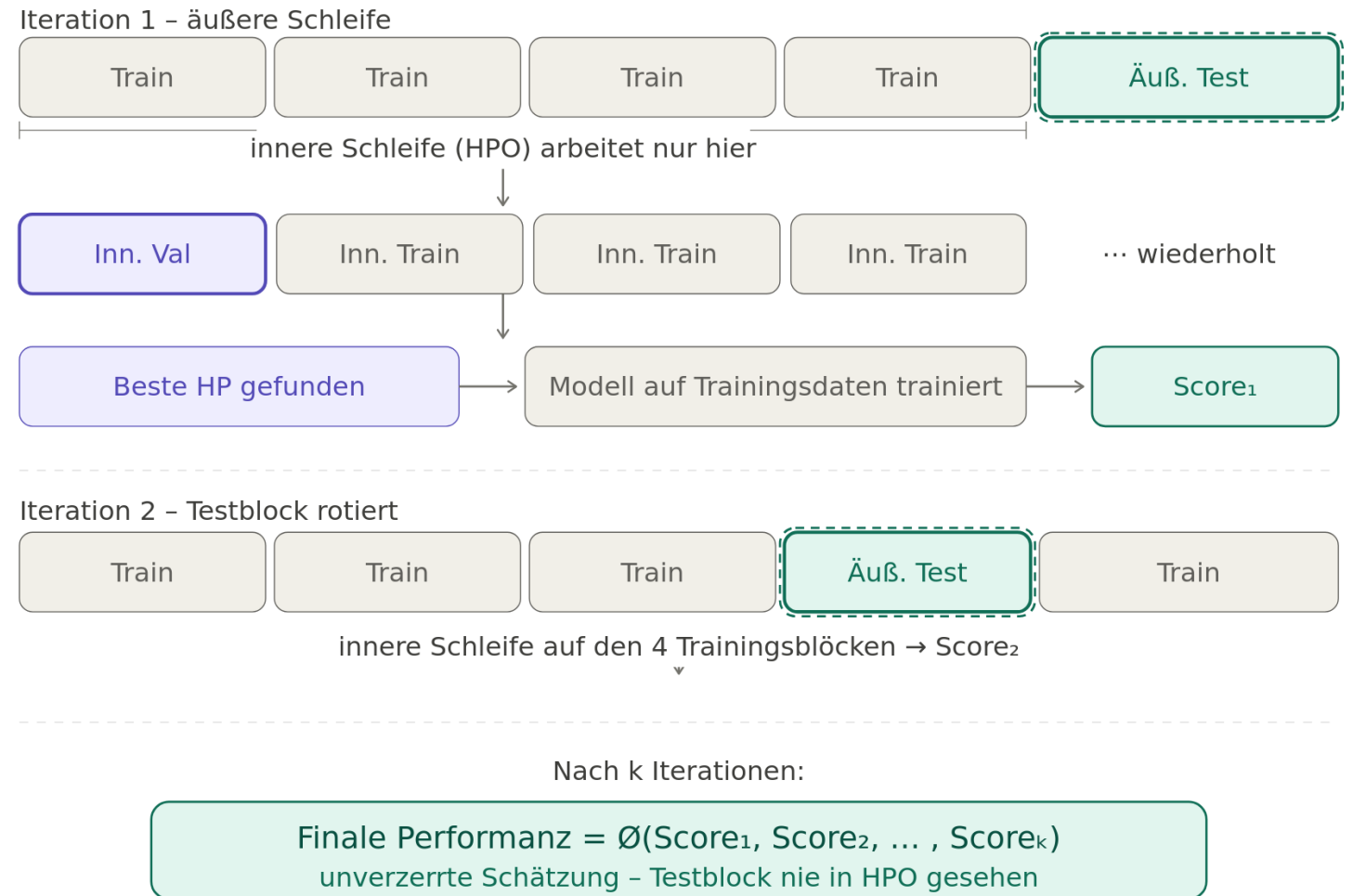
– Problem:

- k-Fold CV und LOOCV nutzen alle Folds für HPO → Modell ist dadurch indirekt auf den Testfold optimiert → indirektes Leakage – optimistische Fehlerrate

– Nested Kreuzvalidierung

– Vorteile:

- Äußere Schleife: Schätzt die generelle Modellperformanz – strikt nur zum Testen
- Innere Schleife: HPO und Modellselektion – sieht den äußeren Testfold nie



EVALUIERUNG

- Bootstrapping
 - Sampling mit Ersatz
 - Vorherige Verfahren wählten Datenpunkte ohne Ersatz
 - Variante: *0.632 bootstrap*
 - Wähle repräsentative Stichprobe
- Ein Datensatz mit n Instanzen wird n mal mit Ersatz gesampelt
- Liefert einen Trainingsdatensatz, welcher n Instanzen hat
- Nutzt einige Instanzen im neuen Datensatz mehrfach
- Ungenutzte Instanzen werden dann im Testdatensatz genutzt

EVALUIERUNG

- Bootstrapping

- Jede Instanz hat eine Wahrscheinlichkeit von:

- $\frac{1}{n}$, um gezogen zu werden

- $1 - \frac{1}{n}$, um nicht gezogen zu werden

- Jedes Item kann n mal gezogen werden

- Wahrscheinlichkeit, dass eine Instanz nicht gezogen wird:

$$\left(1 - \frac{1}{n}\right)^n \approx e^{-1} = 0.368$$

- Trainingsdatensatz

- 63.2% Instanzen

- Testdatensatz

- 36.8% Instanzen

- Einige Instanzen sind mehrfach im Trainingsdatensatz vertreten und ergeben so n Instanzen

EVALUIERUNG

- Bootstrapping
 - Schätzung der Fehlerrate ist pessimistisch
 - 63% Instanzen gegenüber der 90% Instanzen der 10-fold Kreuzvalidierung
 - Idee: Kombiniere des Testdatensatzfehler mit dem Fehler des Models
 - Trainingsfehler normalerweise zu optimistisch
 - Testfehler ist realistischer aber basiert auf wenig Daten
- Bootstrap Fehlerrate:
 - $e = 0,632 e_{test\ instances} + 0.368 \cdot e_{training\ instances}$
 - Wiederhole das bootstrapping mehrmals
 - Berechne den Durchschnitt über alle Wiederholungen

EVALUIERUNG

- Bootstrapping

- Vorteil

- Eine der besten Lösungen für sehr kleine Datensätze

- Nachteile

- Liefert schlechte Ergebnisse für spezielle Fälle
 - Annahme:
 - Zwei Klassen
 - Gleiche Anzahl von Instanzen für jede Klasse
 - Model hat die Trainingsdaten auswendig gelernt und auf neuen Daten geraten (50%)

- $e_{training\ instances} = 0$

- Schätzung der Fehlerrate:
 $0.632 \cdot 50\% + 0.368 \cdot 0\% = 31.6\%$

- Echte Fehlerrate: 50%

EVALUIERUNG

- Vergleich verschiedener Verfahren
 - Simple Methode:
 - Nutze zum Beispiel Kreuzvalidierung
 - Nutze dann das Verfahren mit der geringsten Fehlerrate
 - Problem:
 - Geschätzte Fehlerrate kann aber nicht korrekt sein
 - abhängig von der konkreten Datenaufteilung
 - abhängig von zufälligen Schwankungen in den Folds
 - Wie kann man beurteilen, dass ein Verfahren besser als das andere ist?
- Insbesondere für neue Verfahren muss man dies evaluieren
- Lösung:
 - Statistische Tests, die auf den Konfidenzintervallen basieren

EVALUIERUNG

- Vergleich verschiedener Verfahren
 - Nutzung des t-Tests (auch bekannt als Student's t-test)
 - Zwei Modelle werden auf unabhängigen Datensätzen verglichen
 - Annahme: Daten sind unabhängig & normalverteilt → oft verletzt
 - Wenn der Trainingsdatensatz und der Testdatensatz in beiden Verfahren gleich sind, wird der gepaarte t-Test verwendet
- Wilcoxon Signed-Rank Test
 - keine Normalverteilung nötig
 - basiert auf Rangunterschieden statt Mittelwerten

EVALUIERUNG

- Berechnung der Kosten
 - Vorgestellte Evaluierungen beachten nicht die falsch klassifizierte Daten
 - Das kann zu fragwürdigen Ergebnissen führen!
- Annahme: Ein Ergebnis tritt in 97% der Fälle auf
 - Das Modell sagt dieses Ergebnis immer voraus
 - Ergebnis ist zu 97% korrekt
 - Sind die Fälle, in denen das Ergebnis nicht auftritt vielleicht interessanter?

EVALUIERUNG

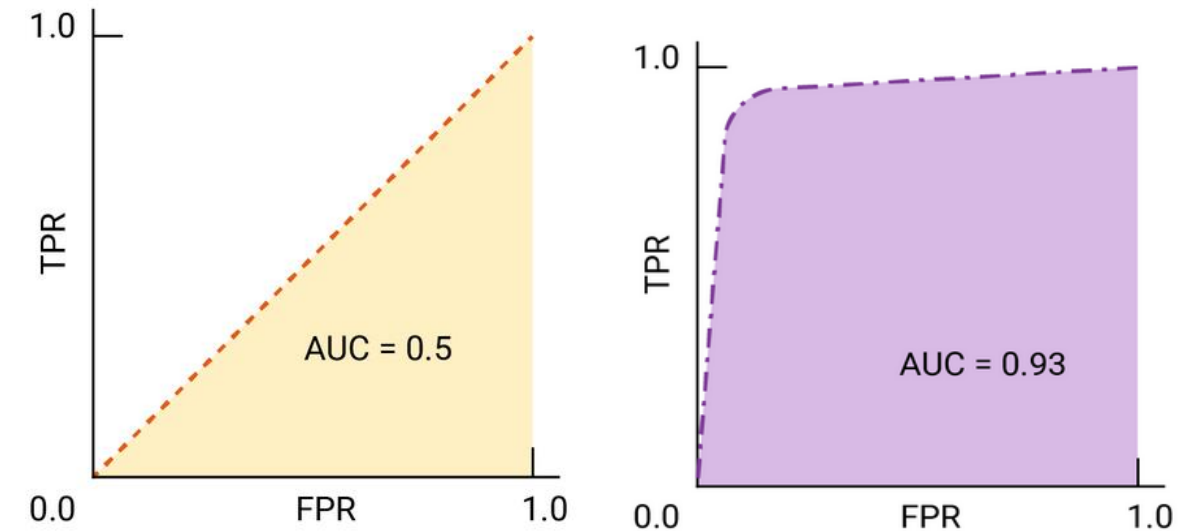
- Precision = $TP / (TP + FP)$
 - Anteil der tatsächlich positiven unter den positiv klassifizierten
- Recall (True Positive Rate) = $TP / (TP + FN)$
 - Anteil der korrekt erkannten positiven Fälle
- False Positive Rate = $FP / (FP + TN)$
 - Anteil der fälschlich als positiv klassifizierten negativen Fälle
- Accuracy = $(TP + TN) / (TP + TN + FP + FN)$
 - Anteil aller korrekt klassifizierten Fälle
- F1-Score = $2 \cdot (Precision \cdot Recall) / (Precision + Recall)$
 - Oft Standard Metrik ist aber problematisch bei unterschiedlichen Kosten

		Vorhergesagte Klasse	
		ja	nein
Tatsächliche Klasse	ja	true positive	false negative
	nein	false positive	true negative

		Predicted condition	
		Cancer	Non-cancer
Actual condition	Total 8 + 4 = 12	7	5
	Cancer 8	6	2
	Non-cancer 4	1	3

EVALUIERUNG

- ROC-Kurve (Receiver Operating Characteristic)
 - Bewertet die Leistung eines Klassifikators über alle möglichen Schwellwerte zur Klassifikation
- Darstellung:
 - x-Achse: False Positive Rate (FPR)
 - y-Achse: True Positive Rate (TPR / Recall)
- Jeder Punkt auf der ROC-Kurve entspricht einem Schwellwert für die Entscheidung, ab welchem Score eine Instanz als positiv klassifiziert wird.



- AUC (Area Under Curve):
 - Fläche unter der ROC-Kurve
 - Maß für die generelle Trennfähigkeit des Modells
- Idealer Punkt: oben links (0,1)
- Zufälliger Klassifikator: Diagonale von (0,0) nach (1,1) (AUC von 0.5)

EVALUIERUNG

- Precision, Recall und F1-Score bei mehreren Klassen
- Macro: Durchschnitt über Klassen, jede Klasse gleich gewichtet
 - Jede Klasse zählt gleich: verzerrt bei unbalancierten Daten
- Micro: Globale Aggregation von TP, FP, FN über alle Datenpunkte
 - Jeder Datenpunkt zählt gleich: stark von häufigen Klassen beeinflusst
- Weighted: Durchschnitt über Klassen, gewichtet nach Klassengröße
 - fair bei unbalancierten Daten

Label	Per-Class F1 Score	Macro-Averaged F1 Score
 Airplane	0.67	$\frac{0.67 + 0.40 + 0.67}{3}$ = 0.58
 Boat	0.40	
 Car	0.67	

Label	True Positive (TP)	False Positive (FP)	False Negative (FN)	Micro-Averaged Values
 Airplane	2	1	1	$\text{Precision} = \frac{6}{6+4} = \mathbf{0.60}$ $\text{Recall} = \frac{6}{6+4} = \mathbf{0.60}$ $\text{F1 Score} = \frac{6}{6 + \frac{1}{2}(4+4)} = \mathbf{0.60}$
 Boat	1	3	0	
 Car	3	0	3	
TOTAL	6	4	4	

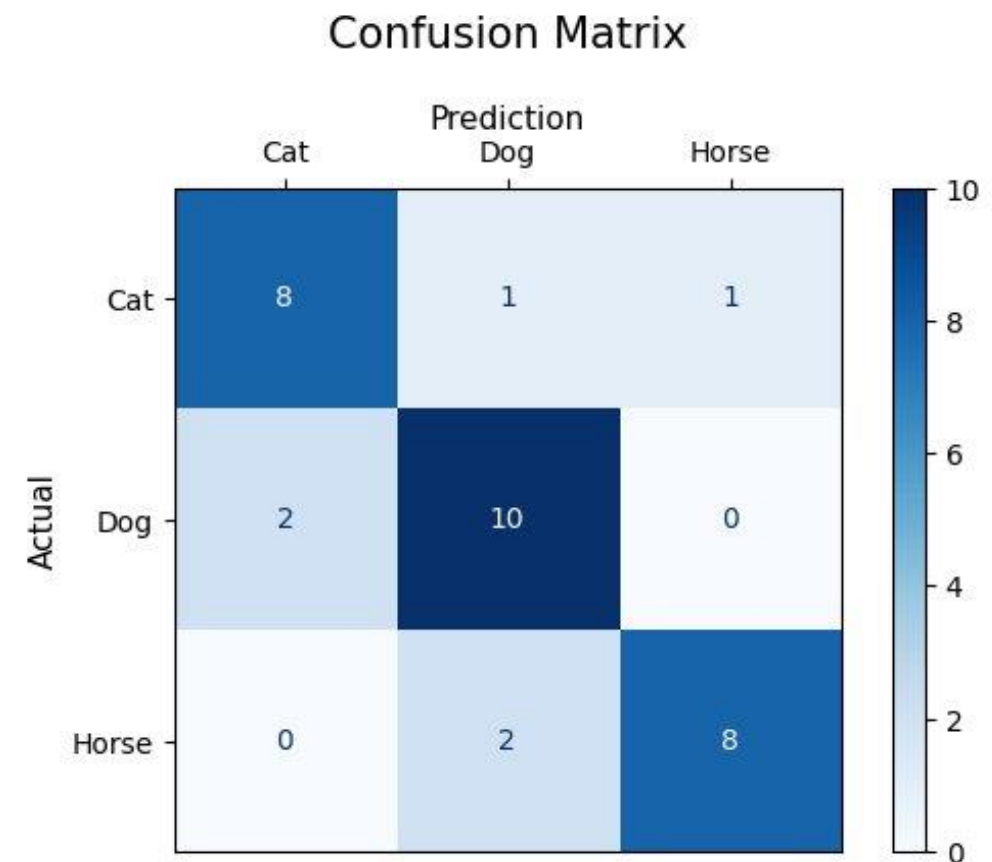
Label	Per-Class F1 Score	Support	Support Proportion	Weighted Average F1 Score
 Airplane	0.67	3	0.3	$(0.67 * 0.3) + (0.40 * 0.1) + (0.67 * 0.6)$ = 0.64
 Boat	0.40	1	0.1	
 Car	0.67	6	0.6	
Total	-	10	1.0	

Bildquelle: <https://towardsdatascience.com/micro-macro-weighted-averages-of-f1-score-clearly-explained-b603420b292f/>

EVALUIERUNG

- Multiklassen Vorhersage: Nutzung einer 2 dimensional *confusion* Matrix
- Jede Zelle repräsentiert die Anzahl der Instanzen mit:
 - Zeile: tatsächliche Klasse
 - Spalte: vorhergesagte Klasse
- Gute Ergebnisse:
 - Große Zahlen auf der Diagonalen
 - Kleine Zahlen außerhalb

- Kann gut gegen Zufallsvorhersagen verglichen werden (Kappa-Statistik)



EVALUIERUNG

- Berechnung der Kosten

- Kappa Statistik: $\frac{p-r}{m-r}$
 - p: Vorhersagen
 - m: Maximal erfolgreiche Vorhersagen
 - r: Zufällige erfolgreiche Vorhersagen
- Maximaler Wert: 100%
- Zufallsvorhersage ergibt 0%
- Die Kappa-Statistik (Cohen's Kappa) bewertet, wie gut ein Modell im Vergleich zu reinem Zufall ist

- Beispiel von Folie 37

- $p = 26 / 32 = 0.8125$ (Diagonale / Alle) Accuracy
- $m = 1$ (100% im best case)
- $r = 10 / 32 * 10 / 32 + 12 / 32 * 13 / 32 + 10 / 32 * 9 / 32 \approx 0.3389$
 - Pro Klasse Zeile * Spalte

- Eingesetzt:
 $(0.8125 - 0.3389) / (1 - 0.3389) = 0.7164$

- Ein Teil der Accuracy kommt durch die Klassenverteilung, aber das Modell ist deutlich besser als Zufall

EVALUIERUNG

- Berechnung der Kosten
 - Kappa Statistik: $\frac{p-r}{m-r}$
 - p: Vorhersagen
 - m: Maximal erfolgreiche Vorhersagen
 - r: Zufällige erfolgreiche Vorhersagen
 - Maximaler Wert: 100%
 - Zufallsvorhersage ergibt 0%
 - Die Kappa-Statistik (Cohen's Kappa) bewertet, wie gut ein Modell im Vergleich zu reinem Zufall ist
- Kann auch genutzt werden um 2 Annotatoren zu vergleichen
- Erweiterung für mehr als 2 Klassen/Annotatoren Fleiss' Kappa
- Kosten sind aber immer noch nicht berücksichtigt!

EVALUIERUNG

- Berechnung der Kosten:
Kostensensitive Klassifikation
 - Nutzung einer Kostenmatrix
 - Jede Zelle repräsentiert die Kosten, welche diese Entscheidung des Klassifikators verursacht
 - Unterschiedliche Zellen können unterschiedliche Kosten haben
- Berechnung der Kosten
 - Summiere alle Zellen der Kostenmatrix für eine Testinstanz auf
 - Kosten werden bei den Vorhersagen ignoriert
 - Kosten werden aber bei der Evaluierung in Betracht gezogen

EVALUIERUNG

- Berechnung der Kosten:
Kostensensitive Klassifikation

- Ergebnisse mit Wahrscheinlichkeiten:
 - Vorhersage der Klasse mit den geringsten Fehlerkosten

- Gegeben:

- Kosten Matrix $\begin{pmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \end{pmatrix}$

- Wahrscheinlichkeiten für die Klassen a, b, c: p_a, p_b, p_c

- Vorhersage a: $1 - p_a$
 - Multipliziere $[p_a \ p_b \ p_c]$ mit $[0 \ 1 \ 1]$:
 $p_b + p_c = 1 - p_a$

- Vorhersage b: $1 - p_b$

- Vorhersage c: $1 - p_c$

- Auswahl der Vorhersage mit der geringsten Fehlerkosten:

- Die meisten Klassifikatoren können angepasst werden, dass sie Wahrscheinlichkeiten mit ausgeben

EVALUIERUNG

- Berechnung der Kosten:
Kostensensitive Klassifikation
- Nutzung der Kostenmatrix während der Trainingsphase
- Ignoriere Kosten bei der Vorhersage
- Simple Methode:
 - Variiere die Größen der Instanzen im Trainingsdatensatz
 - Z.B. Duplikate der Instanzen
- Viele Lernmethoden erlauben gewichtete Instanzen
 - Instanzen werden normalerweise auf 1 initialisiert
 - Instanzen können aber auch über die relativen Kosten der false positives und false negatives initialisiert werden

EVALUIERUNG

- Evaluierung numerischer Vorhersagen
 - Grundideen funktionieren auch hier
 - Messung der Fehlerrate muss angepasst werden:
 - Vorhergesagte Werte in den Testinstanzen: $p_1, \dots, p_n$
 - Tatsächliche Werte: $a_1, \dots, a_n$

EVALUIERUNG

- Evaluierung numerischer Vorhersagen
 - Mittlerer quadratischer Fehler:

$$\frac{\sum_{j=1}^n (p_j - a_j)^2}{n}$$

- Wird sehr häufig genutzt
 - Reagiert empfindlich auf Ausreißer

- Wurzel der mittleren Fehlerquadratsumme:

$$\sqrt{\frac{\sum_{j=1}^n (p_j - a_j)^2}{n}}$$

- Hat die gleiche Dimension wie der vorgesagte Wert
 - Mathematisch unproblematischer

EVALUIERUNG

- Evaluierung numerischer Vorhersagen

- Mittlerer absoluter Fehler:

$$\frac{\sum_{j=1}^n |p_j - a_j|}{n}$$

- Unempfindlich gegen Ausreißer

- Relative Fehler

- Unabhängig von der Größe der Werte

- Nutzung von relativen Fehlern statt der absoluten Fehler in den Berechnungen

EVALUIERUNG

- Evaluierung numerischer Vorhersagen

- Relativer-quadratischer Fehler:

$$\frac{\sum_{j=1}^n (p_j - a_j)^2}{\sum_{j=1}^n (a_j - \bar{a})^2}$$

- Relativ zu den Ergebnissen eines simplen Klassifikators: $\bar{a}$

- $\bar{a}$ ist der mittlere Wert über dem Trainingsdatensatz

- Wurzel des relativen quadratischen Fehler:

$$\sqrt{\frac{\sum_{j=1}^n (p_j - a_j)^2}{\sum_{j=1}^n (a_j - \bar{a})^2}}$$

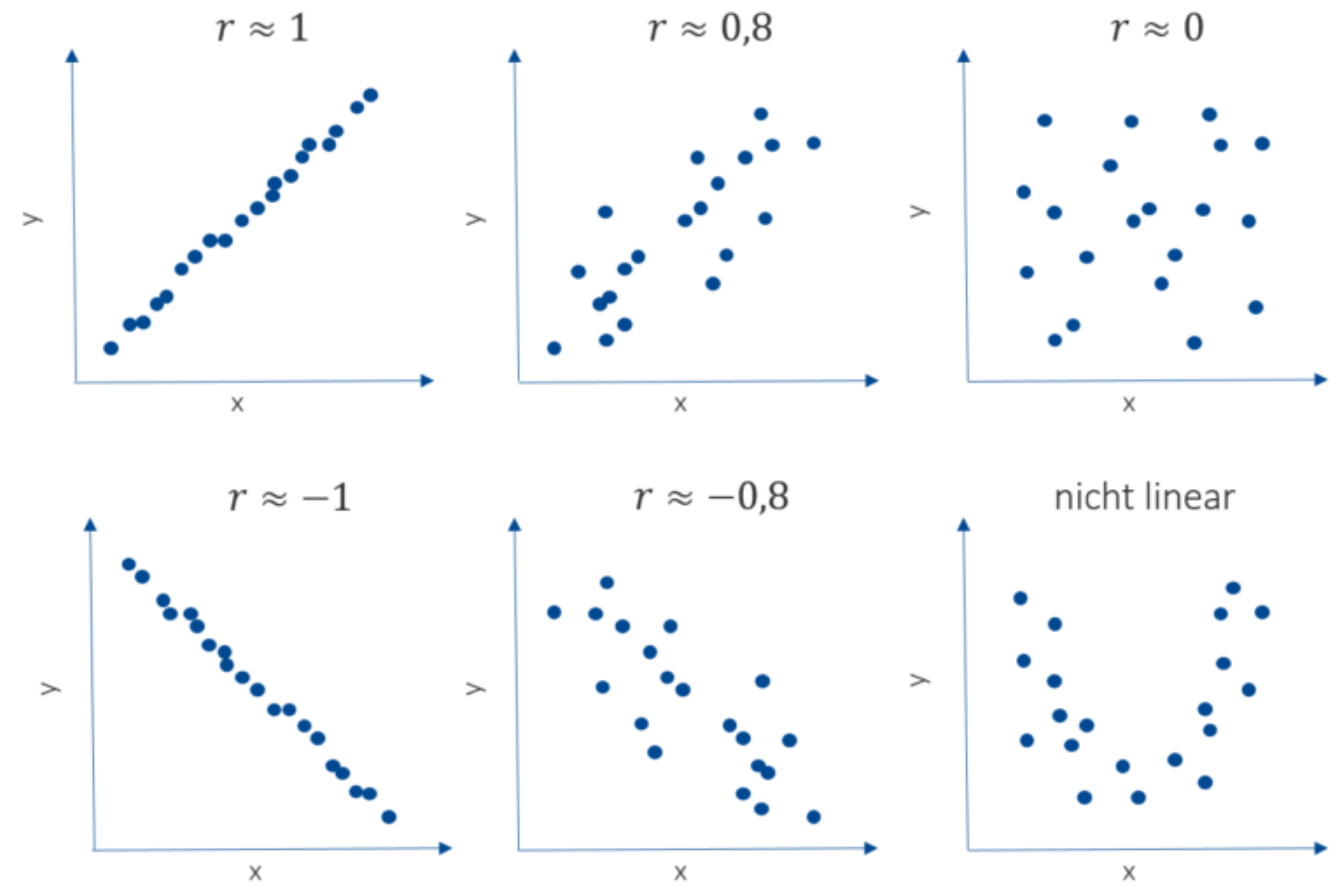
- Analog zum relativen quadratischen Fehler

- Relativer absoluter Fehler:

$$\frac{\sum_{j=1}^n |p_j - a_j|}{\sum_{j=1}^n (a_j - \bar{a})}$$

EVALUIERUNG

- Evaluierung numerischer Vorhersagen: Korrelationskoeffizienten
 - Berechnung der statistischen Korrelation zwischen vorhergesagten und tatsächlichen Wert
 - 1: Perfekte Korrelation
 - 0: Keine Korrelation
 - -1: Perfekte negative Korrelation
 - Unabhängig von der Skalierung, wenn man die vorhergesagten Werten mit einer Konstante multipliziert



EVALUIERUNG

- Pearson-Korrelationskoeffizient für lineare Beziehungen
- SPA= Kovarianz zwischen Vorhersage und tatsächlichen Werten
- SP = Standardabweichung der Vorhersagen
- SA = Standardabweichung der tatsächlichen Werte

$$r = \frac{S_{PA}}{S_P \cdot S_A}$$

$$S_{PA} = \frac{1}{n-1} \sum_{i=1}^n (p_i - \bar{p})(a_i - \bar{a})$$

$$S_P = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (p_i - \bar{p})^2}$$

$$S_A = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (a_i - \bar{a})^2}$$

EVALUIERUNG

- Evaluierung numerischer Vorhersagen
- Sinn des berechneten Maßes hängt von der Anwendung ab:
 - Was soll minimiert werden?
 - Was sind die Kosten der verschiedenen Fehlerarten?

Zusammenfassung

Lernziele

- 1 Warum Evaluierung nötig ist, welche Probleme auftreten und wie Experimente aufgebaut werden
- 2 Warum Datenaufteilung nötig ist sowie Validierungsverfahren und deren Vor- und Nachteile kennen
- 3 Hyperparameter von Modellparametern unterscheiden; HPO-Verfahren kennen; typische Loss-Kurven diagnostizieren
- 4 Klassifikatoren und Regressionsmodelle evaluieren und passende Metriken interpretieren

Themen & Konzepte

Fehlerrate

Training / Validierung / Testdaten

Overfitting

Data Augmentation

Holdout

Stratified Holdout

Repeated Holdout

k-Fold CV

LOOCV

Bootstrapping (0.632)

Grid Search

Random Search

Loss-Kurven

Konfusionsmatrix

Precision / Recall

F1 (Macro/Micro/Weighted)

ROC-Kurve

Kappa-Statistik

Kostenmatrix

MSE / RMSE / MAE

RSE / RAE

Pearson-Korrelation